COMPONENTE CURRICULAR:	Desenvolvimento de Sistemas II
NOME COMPLETO DO ALUNO:	Murillo Ressineti Silva
RA:	10732430

1. Decisão de Projeto e Estrutura

O Bridge: Resolvendo a Explosão de Classes (Problema 1 e 3)

Para o Problema 1, aplicamos o **Bridge** separando a **Abstração** (o que o player faz: gerenciar playlist, volume, interface) da **Implementação** (como o áudio chega: protocolos de transmissão).

- **Lado da Abstração:** Teremos a classe Player (com as variantes Basico, Premium e Studio). Ela possui uma referência para uma interface ProtocoloTransmissao.
- **Lado da Implementação:** A interface ProtocoloTransmissao define métodos como abrirConexao(), receberPacote() e gerenciarBuffer(). As classes concretas são HTTP, WebSocket e P2P.

Por que resolve o Problema 1: Se você mudar o bitrate ou a lógica do protocolo **HTTP**, você altera **uma única classe** (HTTP). A mudança reflete instantaneamente em todos os tipos de Player (Básico, Premium, Studio) porque eles apenas consomem a interface, sem conhecer os detalhes internos do protocolo.

O Decorator: Resolvendo o "Inferno de Booleanos" (Problema 2)

Para o Problema 2, usamos o **Decorator** para envolver o objeto Player com camadas de funcionalidades opcionais.

- **Componente Base:** O Player concreto.
- **Decoradores:** Criamos classes como EqualizadorDecorator, ModoOfflineDecorator e AnaliseAudioDecorator.

Por que resolve o Problema 2: Em vez de 14 atributos booleanos dentro de uma classe gigante, cada funcionalidade vive em sua própria classe. Se o usuário quer equalizador

e modo offline, o sistema faz: new ModoOfflineDecorator(new EqualizadorDecorator(playerPremium)).

Quando o método reproduzir() é chamado, ele percorre a "pilha": o ModoOffline verifica se há cache, o Equalizador aplica o ganho nas frequências e, finalmente, o Player envia o áudio para a saída.

Composição vs. Encadeamento

- **Composição (Bridge):** Ocorre entre o Player e o Protocolo. É um relacionamento "tem um" (*Has-A*). O Player delega a responsabilidade de rede para o objeto injetado.
- **Encadeamento (Decorator):** Ocorre através de uma estrutura de "casca de cebola". Cada decorador implementa a mesma interface do objeto que ele envolve, permitindo que múltiplos comportamentos sejam empilhados de forma transparente para quem chama.

---

2. Análise de Impacto da Mudança (Problema 3)

Processo de Adição do Protocolo QUIC

Atualmente, adicionar um protocolo leva 3 semanas e afeta 12 classes. Na nova arquitetura:

1. **Classes Novas (1):** Cria-se apenas a classe ProtocoloQUIC implementando a interface ProtocoloTransmissao.
2. **Classes Modificadas (0):** Nenhuma classe de Player (Básico, Premium, Studio) ou Decorador precisa ser tocada.
3. **Lógica de Reconexão:** A lógica de "reconexão automática" agora fica centralizada dentro de cada classe de protocolo específica (ou em uma classe base de protocolo), garantindo que o QUIC tenha sua própria regra de *handshake* rápido sem contaminar a lógica do WebSocket ou do HTTP.

Comparação:

- **Cenário Atual:** Relação $N \times M$ (3 players $\times$ 4 protocolos = 12 classes).

- **Nova Arquitetura:** Relação $N + M$ (3 players + 4 protocolos = 7 classes). A complexidade passa a ser aditiva em vez de multiplicativa.

---

3. Limite da Solução

O **Decorator** torna-se inadequado no MackMusic quando a funcionalidade exige **acesso profundo ao estado interno ou ao fluxo de processamento de baixo nível** que não pode ser interceptado de forma "transparente".

Exemplo Específico: Sincronização de Baixa Latência (Codec-Specific)

Se a plataforma decidir implementar um "Modo de Baixa Latência" que precise alterar como o **Codec** decodifica os frames de áudio ou como o **Buffer** de rede é preenchido para evitar *jitter*, o Decorator falha.

Justificativa: O Decorator trabalha na periferia do método (antes ou depois da chamada original). Se a funcionalidade "Modo de Baixa Latência" precisa alterar uma instrução **dentro** do loop de processamento de bits do áudio para otimizar o tempo de resposta, o modelo de "decoração transparente" é quebrado, pois o decorador não consegue "entrar" no meio do algoritmo de reprodução do componente base sem que este seja exposto, o que exigiria refatoração interna (violando o princípio de que o Decorator não altera o objeto decorado).